

VirtuEx 성능 개선 감사 보고서 (2차)

VirtuEx Performance Improvement Audit Report (2nd)

- 감사일: 2026-03-13
- 감사 범위: 프론트엔드/백엔드 전체 성능 관련 기능·이슈 종합 점검
- 감사 방법: 소스 코드 정적 분석, 데이터 흐름 추적, 쿼리 패턴 분석, 번들 구조 검토
- 심각도 기준: Critical(즉시 수정) / High(1주 내) / Medium(2주 내) / Low(개선 권장)
- 비고: 발견 사항 정리만 수행하며, 수정은 포함하지 않음

요약 (Executive Summary)

J. 버그 수정 — 암호화폐 KRW 0원 표시 문제 (수정 완료)

[J-C-01] 소액 암호화폐 KRW 변환 시 0원 표시 (Critical) ☒ 수정 완료

- 파일: frontend/src/lib/format.ts:179-184, 221-223
- 문제점: formatPriceDisplay() 에서 USD → KRW 변환 시 `Math.round(price * exchangeRate)` 사용. SHIB(\$0.00000597)처럼 USD 가격이 매우 낮은 암호화폐의 경우 $0.00000597 \times 1400 \approx 0.00836 \rightarrow \text{Math.round}(0.00836) = 0 \rightarrow \text{formatKRWPrice}(0) = "0 \text{ 원}"$ 표시
- 영향: 대시보드 및 상세 페이지에서 SHIB, GRT 등 소액 암호화폐가 원화 모드에서 "0원"으로 표시
- 수정 내용:
 - `formatKRWPrice()` : 1원 미만 가격에 소수점 표시 추가 (0.01원 이상: 2자리, 0.001원 이상: 3자리, 그 외: 4자리)
 - `formatPriceDisplay()` : `Math.round` 호출을 1원 이상인 경우에만 적용, 1원 미만은 원본 값 유지
 - `formatAmountDisplay()` : 동일한 패턴으로 변동금액도 소수점 유지

A. 프론트엔드 렌더링 성능 — 8건

[A-H-01] 공통 UI 컴포넌트 React.memo 미적용 (High)

- 파일:

- frontend/src/components/ui/Tabs.tsx — 탭 전환 컴포넌트
- frontend/src/components/ui/Pagination.tsx — 페이지네이션
- frontend/src/components/ui/ConfirmModal.tsx — 확인 모달
- frontend/src/components/ui/ContentModal.tsx — 콘텐츠 모달
- frontend/src/components/ui/ToastContainer.tsx — 토스트 알림
- frontend/src/components/layout/Footer.tsx — 푸터

- 설명: 프로젝트 전반에서 사용되는 공통 UI 컴포넌트들이 React.memo 미적용. 부모 컴포넌트 상태 변경 시 props가 변경되지 않았음에도 불필요하게 리렌더링됨. 특히 Tabs와 Pagination은 데이터 목록 페이지에서 빈번하게 리렌더링 발생

- 권장: React.memo() 래핑

[A-H-02] JSX props 인라인 함수 생성으로 인한 리렌더링 (High)

- 파일:

- frontend/src/components/ui/Pagination.tsx:88, 115, 122, 135, 149, 156 — 6개 onClick 인라인
- frontend/src/components/ui/Tabs.tsx:51, 72 — 탭별 인라인 함수
- frontend/src/components/ui/RichEditor.tsx:142-172 — TipTap 톨바 버튼 10개+ 인라인
- frontend/src/components/market/AssetList.tsx:257-283 — 카테고리/정렬 버튼 인라인
- frontend/src/components/chat/InviteModal.tsx:156 — 검색 onChange 인라인
- frontend/src/components/chat/CreateRoomModal.tsx:128, 158 — 입력 onChange 인라인
- frontend/src/components/auth/PhoneVerification.tsx:128, 152 — 전화번호 입력 인라인
- frontend/src/components/alerts/PriceAlertModal.tsx:198 — 가격 입력 인라인

- 설명: JSX props에서 onClick={() => ...} 패턴으로 매 렌더 시 새 함수 참조 생성. React.memo 된 자식 컴포넌트에 전달 시 memo 효과 무효화. 전체 프론트엔드에서 30개 이상 발견

- 권장: useCallback 으로 핸들러 추출 또는 React.memo 미적용 컴포넌트에서는 현행 유지

[A-H-03] PortfolioAnalytics 정렬 연산 미메모이제이션 (High)

- 파일: frontend/src/components/portfolio/PortfolioAnalytics.tsx:706-777

- 설명: 자산 성과 섹션에서 holdings 배열을 .sort((a, b) => b.pnlPercent - a.pnlPercent) 로 정렬하나 useMemo 미적용. 매 렌더마다 정렬 재실행. 보유 종목 50개 이상 시 체감 지연 가능

- 권장: useMemo 로 정렬 결과 캐싱

[A-M-01] 인라인 스타일 객체 생성 (Medium)

- 파일:

- frontend/src/components/portfolio/PortfolioAnalytics.tsx:365-371, 804-811, 877-887 — 차트 tooltip contentType
- frontend/src/components/trading/OrderBook.tsx:364-411 — 호가창 depth bar style

- 설명: Recharts 컴포넌트의 contentType, style props에 인라인 객체 리터럴 전달. 매 렌더마다 새 객체 참조 생성으로 불필요한 diff 발생

- 권장: 컴포넌트 외부 또는 useMemo 로 스타일 객체 추출

[A-M-02] Zustand 다중 구독 리렌더링 (Medium)

- 파일:

- `frontend/src/components/layout/MainContent.tsx:15-16` — `useChatStore` 2회 호출 (`isPinned`, `isOpen`)
- `frontend/src/components/chat/MessageArea.tsx:73-99` — `useChatStore` 4개 필드 개별 구독
- `frontend/src/components/layout/Header.tsx:30-36` — `useAuthStore` + `useSettingsStore` 다중 필드
- 설명: Zustand 스토어에서 개별 필드를 각각 구독하면 어느 하나만 변경되어도 모두 리렌더링 트리거. 특히 Header는 모든 페이지에서 렌더링되므로 영향 범위가 넓음
- 권장: 단일 선택터로 필요한 필드를 한 번에 추출 (`shallow` 비교 사용)

[A-M-03] OrderBook 가시 영역 슬라이싱 미메모이제이션 (Medium)

- 파일: `frontend/src/components/trading/OrderBook.tsx:303-310`
- 설명: 호가 데이터의 가시 영역 계산(슬라이싱)이 `useMemo` 외부에서 수행되어 매 렌더마다 재계산
- 권장: 슬라이싱 로직을 기존 `useMemo` 내부로 이동

[A-L-01] Footer 정적 데이터 매 렌더 재생성 (Low)

- 파일: `frontend/src/components/layout/Footer.tsx`
- 설명: 기술 스택 아이콘 배열 등 정적 데이터가 컴포넌트 내부에서 매 렌더마다 재생성. `React.memo` 미적용과 결합 시 불필요한 연산
- 권장: 모듈 스코프 상수로 추출 또는 `useMemo` 적용

[A-L-02] 차트 컨피그 인라인 재생성 (Low)

- 파일: `frontend/src/components/chart/CandlestickChart.tsx`
- 설명: Recharts 차트 설정 객체(그리드, 축 설정 등)가 컴포넌트 내부에서 매 렌더마다 생성
- 권장: 정적 설정은 모듈 스코프로 이동

B. 프론트엔드 데이터 페칭 — 7건

[B-C-01] React Query 기본 `staleTime`이 5초로 과도하게 짧음 (Critical)

- 파일: `frontend/src/components/layout/QueryProvider.tsx:76-80`
- 설명: 전역 기본 `staleTime: 5 * 1000` (5초). 창 포커스 시 5초 이상 지난 모든 쿼리가 자동 `refetch`됨. 탭 전환만으로도 대량 API 호출 발생. 사용자 설정, 프로필 등 변경 빈도가 낮은 데이터도 5초마다 `stale` 처리
- 영향: 사용자가 다른 탭에서 돌아올 때마다 모든 활성 쿼리가 동시 `refetch` → API 서버 부하 급증
- 권장: 기본 `staleTime`을 30초~60초로 상향, 실시간성이 필요한 쿼리만 개별 설정

[B-H-01] 10개 이상 React Query 훅에 `staleTime` 미설정 (High)

- 파일:
- `frontend/src/hooks/usePortfolio.ts:112` — 포트폴리오 요약 (`refetchInterval: 10s`, `staleTime` 없음)
- `frontend/src/hooks/usePortfolio.ts:130` — 포트폴리오 평가 (`refetchInterval: 10s`, `staleTime` 없음)

- `frontend/src/hooks/useChat.ts:37` — 채팅방 목록 (`refetchInterval: 30s`, `staleTime` 없음)
- `frontend/src/hooks/useOrders.ts:73` — 체결 내역 (`refetchInterval: 10s`, `staleTime` 없음)
- `frontend/src/hooks/useOrders.ts:199` — 최근 체결 (`refetchInterval: 10s`, `staleTime` 없음)
- `frontend/src/hooks/useLeaderboard.ts:69` — 리더보드 (`refetchInterval: 10s`, `staleTime` 없음)
- `frontend/src/hooks/useNews.ts:36` — 뉴스 목록 (`refetchInterval: 60s`, `staleTime` 없음)
- `frontend/src/hooks/useAdmin.ts:533` — 관리자 알림 (`refetchInterval: 30s`, `staleTime` 없음)
- `frontend/src/hooks/useActivityFeed.ts:57` — 활동 피드 (`refetchInterval: 30s`, `staleTime` 없음)
- 설명: `refetchInterval` 은 설정되어 있으나 `staleTime` 이 없어, 전역 기본값(5초)이 적용됨. 창 포커스/컴포넌트 마운트 시 stale 판정되어 `refetchInterval`과 무관하게 즉시 `refetch` 발생
- 영향: 리더보드 10초 폴링 + 포트폴리오 10초 폴링 + 주문 10초 폴링 = 매 10초 최소 3개 API 호출, 사용자 수 × 3
- 권장: 각 훅에 적절한 `staleTime` 설정 (예: 리더보드 30초, 뉴스 120초)

[B-H-02] REST 폴링과 WebSocket 이중 데이터 수신 (High)

- 파일: `frontend/src/hooks/useMarket.ts:31, 34`
- 설명: `useMarketPrices()` 가 `refetchInterval: 30_000` (30초)로 REST 폴링하면서 동시에 WebSocket으로 실시간 가격도 수신. WebSocket이 정상 작동 중이면 REST 폴링은 불필요한 중복 호출
- 권장: WebSocket 연결 상태에 따라 REST 폴링 활성화/비활성화 조건 분기

[B-M-01] 포트폴리오 2개 훅 중복 요청 (Medium)

- 파일: `frontend/src/hooks/usePortfolio.ts:112, 130`
- 설명: `usePortfolioSummary()` 와 `usePortfolioValuation()` 두 훅이 각각 10초마다 별도 API 호출. 둘 다 포트폴리오 데이터를 조회하며 상당 부분 중복
- 권장: 단일 API로 통합하거나, 하나의 훅이 다른 훅의 데이터에 의존하도록 설계

[B-M-02] 상세 페이지 워터폴 요청 (Medium)

- 파일: `frontend/src/app/(main)/asset/[symbol]/page.tsx`
- 설명: 자산 상세 페이지에서 `useAssetPrice`, `useCandlesticks`, `useOrderBook`, `useRecentTrades` 4개 훅이 독립적으로 실행되나, React Query가 컴포넌트 마운트 후 순차적으로 시작하여 워터폴 패턴 발생 가능
- 권장: 프리페치 또는 서버 컴포넌트에서 초기 데이터 로드

[B-M-03] 프리페치 전략 부재 (Medium)

- 파일: 프론트엔드 전체
- 설명: `queryClient.prefetchQuery()` 사용 사례 없음. 사용자가 대시보드에서 종목을 클릭할 때 상세 페이지 데이터 프리페치 미적용. Link 호버 시 데이터 선로딩 미구현
- 권장: 주요 네비게이션 경로에 프리페치 적용 (종목 상세, 포트폴리오 등)

[B-L-01] 캔들스틱 클라이언트 사이드 집계 (Low)

- 파일: `frontend/src/hooks/useMarket.ts:104-134`

- **설명:** `aggregateCandles()` 함수가 1분봉 → 상위 간격 집계를 클라이언트에서 수행. 새 Map 생성, `Array.from()`, `Math.max/min` 루프 등 매 쿼리마다 실행. 데이터량이 많을 경우 프레임 드롭 가능
- **권장:** 서버에서 간격별 캔들스틱 직접 제공 (이미 구현됨 — 클라이언트 폴백 코드 제거 검토)

C. 프론트엔드 번들/빌드 — 3건

[C-H-01] 차트/에디터 라이브러리 정적 임포트 (High)

- **파일:**
 - `frontend/src/components/portfolio/PortfolioAnalytics.tsx` — Recharts 다수 컴포넌트 정적 임포트
 - `frontend/src/components/ui/RichEditor.tsx` — TipTap 에디터 라이브러리 (~150KB+)
- **설명:** Recharts와 TipTap은 큰 번들 사이즈를 가진 라이브러리. 포트폴리오 분석과 커뮤니티 글 작성 등 특정 페이지에서만 사용됨에도 정적 임포트되어 초기 번들에 포함됨
- **참고:** `CandlestickChart`, `Ailnsights`, `RichEditor`(community/new)는 이미 `next/dynamic` 적용 완료
- **권장:** `PortfolioAnalytics`의 차트 영역을 동적 임포트로 분리

[C-M-01] Next.js 정적 자산 캐시 헤더 미설정 (Medium)

- **파일:** `frontend/next.config.ts:28-42`
- **설명:** 보안 헤더는 모든 라우트에 적용되나, 정적 자산(JS/CSS/이미지)에 대한 장기 캐시 헤더(`Cache-Control: public, max-age=31536000, immutable`) 미설정. Next.js가 `_next/static` 경로에는 자동 적용하나, `public/` 디렉토리 자산은 캐시 정책 없음
- **권장:** `public/` 자산에 캐시 헤더 추가

[C-L-01] API Rewrite 추가 흠 (Low)

- **파일:** `frontend/next.config.ts:20-26`
- **설명:** `/api/:path*` → 백엔드 URL로 rewrite. 클라이언트 → Next.js 서버 → 백엔드로 2-hop 구조. 서버 컴포넌트에서 직접 백엔드 호출 시에도 rewrite 경유
- **권장:** 서버 컴포넌트에서는 백엔드 직접 호출 검토

D. 백엔드 DB 쿼리 — 7건

[D-C-01] 공지사항 알림 발송 시 전체 사용자 무제한 조회 (Critical)

- **파일:** `user-auth/src/application/services/announcement.service.ts:242-245, 291-294`
- **설명:** 공지사항 생성/수정 시 `findMany({ where: { isActive: true } })` 로 전체 활성 사용자를 한 번에 메모리에 로드한 후 알림 생성. 사용자 10,000명 이상 시 메모리 폭증 + DB 타임아웃 위험
- **권장:** 1,000명 단위 배치 처리 (cursor 기반)

[D-H-01] 전략 댓글 무제한 로드 (High)

- 파일: `user-auth/src/presentation/controllers/strategy.controller.ts:129-131`
- 설명: `getStrategy()` 호출 시 `include: { comments: { orderBy: ... } }` 로 해당 전략의 모든 댓글을 한 번에 로드. 댓글 10,000건 이상 시 응답 크기 폭증
- 권장: 댓글 페이지네이션 분리

[D-H-02] 일일 손실 계산 N+1 패턴 (High)

- 파일: `order-engine/src/application/services/order.service.ts:122-150`
- 설명: 당일 매도 거래 목록 조회 후, 각 종목별로 매수 거래를 개별 조회하는 N+1 패턴. 10개 종목 매도 시 11개 쿼리 실행
- 권장: JOIN 또는 배치 조회로 통합

[D-H-03] 통계 대시보드 전체 사용자 로드 (High)

- 파일: `user-auth/src/presentation/controllers/statistics.controller.ts:111-115, 138-145`
- 설명: 등록 통계 조회 시 기간 내 전체 사용자 레코드를 메모리에 로드. 1년 통계 조회 시 수만 건 로드 가능
- 권장: GROUP BY 집계 쿼리 또는 날짜별 카운트로 변경

[D-M-01] 가격 히스토리 limit 상한 미검증 (Medium)

- 파일: `market-data/src/application/services/market-data.service.ts:188-193`
- 설명: `getPriceHistory(symbol, limit)` 에서 클라이언트가 전달한 `limit` 값에 상한 검증 없음. `limit=100000` 요청 시 대규모 DB 스캔 발생
- 권장: 최대 limit 1000 등 상한 적용

[D-M-02] 커뮤니티 좋아요 테이블 userId 단독 인덱스 부재 (Medium)

- 파일: `user-auth/prisma/schema.prisma`
- `CommunityPostLike` (line 310-321) — `@@unique([userId, postId])` 복합만 존재
- `CommunityCommentLike` (line 388-399) — `@@unique([userId, commentId])` 복합만 존재
- `CommunityStrategyLike` (line 361-371) — `@@unique([userId, strategyId])` 복합만 존재
- 설명: "사용자가 좋아요한 게시물 목록" 등 `userId` 단독 필터 쿼리 시 복합 인덱스의 선행 컬럼이 아닌 경우 풀 테이블 스캔 발생 가능
- 권장: `@@index([userId])` 추가

[D-L-01] 관리자 사용자 목록 인메모리 정렬 (Low)

- 파일: `user-auth/src/application/services/admin.service.ts:91-95`
 - 설명: DB에서 페이지네이션된 결과를 가져온 후 역할 우선순위로 인메모리 재정렬. 페이지 2의 데이터가 실제 정렬 순서와 불일치할 수 있음
 - 권장: DB ORDER BY에 역할 우선순위 CASE 식 적용
-

E. 백엔드 메모리/캐시 — 5건

[E-C-01] PriceHistory 테이블 무한 성장 (Critical)

- 파일: `market-data/prisma/schema.prisma:30-31`
- 설명: 스키마 주석에 `TOD0: Production - apply timestamp-based 90-day retention` 기재되어 있으나 미구현. 매 10초마다 전체 자산 가격이 DB에 저장되어 일 864,000건 (50자산 × 8,640회) 누적. 1년 후 3억 건 이상
- 권장: 90일 이전 데이터 삭제 크론잡 구현

[E-C-02] PageView 테이블 무한 성장 (Critical)

- 파일: `user-auth/prisma/schema.prisma:222-233`
- 설명: `TOD0: Production - apply 30-day data retention` 미구현. 활성 사용자 1,000명 × 일 10 PV = 일 10,000건, 1년 후 365만 건 누적. 통계 쿼리 성능 저하
- 권장: 30일 이전 데이터 삭제 크론잡 구현

[E-H-01] ReadReceipt 테이블 TTL 없음 (High)

- 파일: `chat/prisma/schema.prisma:67-78`
- 설명: 메시지 읽음 확인 레코드에 만료 정책 없음. 채팅 메시지가 참가자 수만큼 레코드 생성. 월 100만 메시지 × 평균 3명 = 월 300만 건 누적
- 권장: 90일 이전 읽음 확인 삭제 또는 집계 테이블로 이관

[E-M-01] 포트폴리오 서비스 시장 가격 캐시 없음 (Medium)

- 파일: `portfolio/src/domain/services/balance.service.ts:756-824`
- 설명: `getPortfolioValuation()` 호출마다 `fetchMarketPrices()` 로 `market-data` 서비스에 HTTP 요청. 리더보드 계산 시 모든 사용자에 대해 동일 심볼 가격을 반복 조회
- 권장: 가격 데이터 로컬 캐시 (TTL 10초) 적용

[E-M-02] 환율 캐시 로직 2개 서비스에 중복 (Medium)

- 파일:
 - `order-engine/src/application/services/order.service.ts:49-105` — 환율 캐시 10분 TTL
 - `portfolio/src/domain/services/balance.service.ts:69-121` — 동일 환율 캐시 로직 복제
- 설명: 두 서비스가 동일한 환율 조회 + 캐시 로직을 각각 구현. 코드 중복 + API 환율 서비스 이중 호출 가능성
- 권장: 공통 모듈로 추출하거나 API Gateway 레벨에서 환율 캐싱

F. 백엔드 동시성/처리 — 4건

[F-H-01] 캔들스틱 업데이트 개별 SQL 실행 (High)

- 파일: `market-data/src/application/services/market-data.service.ts:428-451`

- **설명:** `updateCandlesticks()` 에서 각 자산별로 `INSERT ... ON CONFLICT` SQL을 개별 실행. 50개 자산 기준 매 분 50개 순차 SQL. 해당 처리는 60틱(1분)마다 실행

- **권장:** 멀티 행 INSERT 또는 PostgreSQL COPY로 배치 처리

[F-H-02] 상위 간격 캔들스틱 집계 이중 쿼리 (High)

- **파일:** `market-data/src/application/services/market-data.service.ts:458-544`

- **설명:** `aggregateHigherIntervals()` 에서 4개 간격(5m, 15m, 1h, 4h) × N개 심볼에 대해 각각 `groupBy` + `queryRaw` 2개 쿼리 실행. 50심볼 기준 $4 \times 50 \times 2 = 400$ 개 쿼리/분

- **권장:** 단일 SQL로 간격별 집계 통합

[F-M-01] 정산 복구 배치 크기 50건 고정 (Medium)

- **파일:** `order-engine/src/application/services/settlement-recovery.service.ts:55-59`

- **설명:** 미정산 건을 50건씩 조회 후 병렬 처리, 30초 대기 후 다음 배치. 미정산 백로그 10,000건 시 $10,000/50 \times 30$ 초 $\approx$ 100분 소요

- **권장:** 배치 크기 동적 조절 또는 대기 시간 축소

[F-M-02] Binance API 병렬 배치 크기 10 고정 (Medium)

- **파일:** `market-data/src/application/services/market-data.service.ts:297-329`

- **설명:** Binance 과거 가격 조회를 10개씩 병렬 처리. 30개 암호화폐 기준 3배치 필요. 각 fetch에 타임아웃 미설정으로 느린 응답 시 전체 배치 지연

- **권장:** `AbortSignal.timeout(5000)` 추가 + 배치 크기 환경변수화

G. WebSocket/실시간 — 6건

[G-H-01] 가격 브로드캐스트 메시지 배칭 없음 (High)

- **파일:** `api-gateway/src/gateway/price.gateway.ts:166-172`

- **설명:** `broadcastPrice()` 가 각 심볼별로 개별 emit. 50개 자산 × 매초 = 초당 50회 브로드캐스트. Socket.IO의 내부 직렬화 오버헤드가 50배 발생

- **권장:** 다중 심볼 가격을 단일 메시지로 배칭 (예: `prices:batch` 이벤트)

[G-H-02] 채팅 게이트웨이 `joinUserToRoom` `fetchSockets` 비효율 (High)

- **파일:** `api-gateway/src/gateway/chat.gateway.ts:211-222`

- **설명:** `joinUserToRoom()` 에서 `server.in(socketId).fetchSockets()` 비동기 호출로 소켓 조회 후 `join()` 수행. `fetchSockets()` 는 어댑터 단 조회로 소켓 ID를 이미 알고 있음에도 불필요한 양복 발생

- **권장:** `server.sockets.sockets.get(socketId)?.join(roomId)` 직접 접근

[G-M-01] 채팅 이벤트 미들웨어 전 이벤트 실행 (Medium)

- 파일: `api-gateway/src/gateway/chat.gateway.ts:48-62`

- 설명: `socket.onAny()` 미들웨어가 모든 수신 이벤트(join, leave, typing, ping 포함)에서 `content` 필드 길이를 검사. 채팅 메시지 외 이벤트에서도 불필요한 검사 수행

- 권장: 메시지 관련 이벤트만 필터링

[G-M-02] 가격 알림 확인 매 틱 실행 (Medium)

- 파일: `api-gateway/src/gateway/price-subscriber.service.ts:157-194`

- 설명: `checkPriceAlerts()` 가 가격 틱마다 호출되어 해당 심볼의 모든 활성 알림을 순회. 50개 자산 x 초당 1틱 x 심볼당 100개 알림 = 초당 5,000회 비교 연산

- 권장: 알림을 가격 범위별 버킷으로 인덱싱하여 O(1) 비교

[G-M-03] WebSocket 하트비트 타임아웃 5초로 짧음 (Medium)

- 파일: `frontend/src/hooks/useWebSocket.ts:22`

- 설명: `PONG_TIMEOUT = 5_000ms` — 느린 네트워크(3G, 해외 VPN 등)에서 5초 내 pong 미수신 시 연결 종료 + 재연결. 불필요한 재연결 빈번 발생 가능

- 권장: 10초로 상향 검토

[G-L-01] 채팅 타이핑 타이머 고빈도 생성 (Low)

- 파일: `frontend/src/hooks/useChatSocket.ts:61-77`

- 설명: `setTypingUser()` 호출마다 3초 자동 제거 타이머 생성. 빠른 타이핑 시 초당 5-10회 타이머 생성/클리어 반복. 대규모 채팅방에서 다수 사용자 타이핑 시 타이머 오버헤드

- 권장: 디바운스 적용 (마지막 키 입력 후 3초)

H. 가격 엔진/마켓 데이터 — 5건

[H-H-01] 가격 엔진 6개 Map 개별 관리 (High)

- 파일: `market-data/src/domain/services/price-engine.service.ts:30-35`

- 설명: `prices`, `openPrices24h`, `high24h`, `low24h`, `volumes`, `volatilityMultipliers` — 6개 Map이 심볼을 키로 개별 관리. `generateTick()` 마다 6회 Map 조회 + 6회 Map 업데이트. 50자산 x 초당 = 초당 600회 Map 연산

- 권장: 단일 Map 구조체로 통합

[H-H-02] Binance WebSocket JSON 파싱 에러 무시 (High)

- 파일: `market-data/src/domain/services/binance-price.service.ts:112-114`

- 설명: Binance WebSocket 메시지 `JSON.parse()` 실패를 빈 catch로 완전 무시. Binance 데이터 형식 변경 시 모든 가격 업데이트가 조용히 실패하며, 시스템은 GBM 시뮬레이션으로 폴백되어 실제 가격과 괴리 발생

- **권장:** 파싱 실패 카운터 + 임계값 초과 시 경고 로깅

[H-M-01] Binance 재연결 지수 백오프 상한 30초 (Medium)

- **파일:** `market-data/src/domain/services/binance-price.service.ts:184-188`
- **설명:** 재연결 딜레이가 5초→10초→20초→30초(상한)로 증가. Binance 장기 장애 시 30초마다 재연결 시도가 반복되어 불필요한 네트워크 트래픽 + 에러 로깅 발생
- **권장:** 상한을 5분으로 상향, 장기 장애 시 알림 발송

[H-M-02] 가격 캐시 TTL 10초로 짧음 (Medium)

- **파일:** `market-data/src/infrastructure/redis/price-cache.service.ts:40-43`
- **설명:** Redis 가격 캐시 TTL이 10초. 가격 엔진이 1초마다 갱신하므로 정상 상황에서는 문제없으나, 엔진 중단 시 10초 후 모든 가격 캐시 만료 → 프론트엔드에 빈 데이터 반환
- **권장:** TTL을 60초로 상향, 마지막 유효 가격을 더 오래 유지

[H-L-01] roundPrice() 틱당 3회 호출 (Low)

- **파일:** `market-data/src/domain/services/price-engine.service.ts:231-235`
- **설명:** `generateTick()` 에서 `newPrice` , `bid` , `ask` 각각에 `roundPrice()` 호출. 가격 크기 조건 분기 3회 × 50자산 = 초당 150회. 경미한 오버헤드
- **권장:** 단일 호출 후 결과 재사용

I. API 게이트웨이/네트워크 — 4건

[I-H-01] 프록시 전체 서비스 동일 5초 타임아웃 (High)

- **파일:** `api-gateway/src/proxy/proxy.service.ts:62`
- **설명:** 모든 마이크로서비스 프록시 호출에 동일한 5초 타임아웃 적용. 리더보드 계산(사용자 정보 보강 포함), 통계 집계 등 시간이 걸리는 API도 5초 제한. 복잡한 쿼리가 5초 초과 시 서킷 브레이커 카운트 증가 → 잘못된 서비스 차단
- **권장:** 엔드포인트별 타임아웃 설정 (읽기 5초, 집계 15초 등)

[I-M-01] 서킷 브레이커 쿨다운 30초 (Medium)

- **파일:** `api-gateway/src/proxy/proxy.service.ts:24-25`
- **설명:** 서킷 브레이커 임계값 5회, 쿨다운 30초. 5초 타임아웃 × 5회 = 25초 후 서킷 오픈, 30초 대기 후 반개방. 총 복구 시간 55초. 일시적 네트워크 이슈로도 55초간 서비스 차단 가능
- **권장:** 반개방 상태에서 점진적 복구 (10%, 25%, 50%, 100%) 검토

[I-M-02] 전역 TtlCacheInterceptor 매 요청 메타데이터 조회 (Medium)

- **파일:** `api-gateway/src/app.module.ts:51` , `api-gateway/src/config/cache.interceptor.ts:50-57`

- **설명:** `TtlCacheInterceptor` 가 `APP_INTERCEPTOR` 로 글로벌 등록되어 모든 요청(POST, PUT, DELETE 포함)에서 Reflector 메타데이터 조회 실행. `@CacheTTL` 데코레이터가 없는 대다수 핸들러에서도 불필요한 메타데이터 록업 발생
- **권장:** GET 메서드 초기 필터링을 메타데이터 조회 이전에 배치

[I-L-01] 압축 임계값 1KB로 낮음 (Low)

- **파일:** `api-gateway/src/main.ts:35`
- **설명:** `compression({ threshold: 1024 })` — 1KB 이상 응답 모두 압축. 1-2KB 수준의 작은 응답은 압축 오버헤드(CPU + 시간)가 절감 효과보다 클 수 있음
- **권장:** 임계값 4KB~8KB로 상향 검토

검증 결과

1차 대비 개선 현황

카테고리	Critical	High	Medium	Low	합계
A. 프론트엔드 렌더링	0	3	3	2	8
B. 프론트엔드 데이터 페칭	1	2	3	1	7
C. 프론트엔드 번들/빌드	0	1	1	1	3
D. 백엔드 DB 쿼리	1	3	2	1	7
E. 백엔드 메모리/캐시	2	1	2	0	5
F. 백엔드 동시성/처리	0	2	2	0	4
G. WebSocket/실시간	0	2	3	1	6
H. 가격 엔진/마켓 데이터	0	2	2	1	5
I. API 게이트웨이/네트워크	0	1	2	1	4
J. 버그 수정 (0원 표시)	1	0	0	0	1
합계	5	17	20	8	50
항목	상태				

프론트엔드 빌드	✅ 성공	
0원 표시 버그 수정 검증	✅ formatKRWPrice 소수점 표시 확인	
백엔드 타입 검사	✅ 전체 서비스 성공	
서비스 기동	✅ 전체 정상	
구분	1차 감사	2차 감사
총 발견 건수	12건 (전체 수정 완료)	50건 (수정 미진행, 보고만)
버그 수정	-	1건 수정 완료 (0원 표시)
Critical	1	5
High	5	17
Medium	4	20
Low	2	8

범위주요 병목 집중전체 코드베이스 심층 감사

2차 감사에서 새롭게 발견된 주요 영역

- **프론트엔드**: React Query staleTime 설정 부재 (10개+ 혹), 공통 UI 컴포넌트 memo 미적용
- **백엔드**: 데이터 보존 정책 미구현 (PriceHistory/PageView/ReadReceipt 무한 성장)
- **가격 엔진**: 6개 Map 분산 관리, 캔들스틱 집계 쿼리 폭증 (분당 400개)
- **네트워크**: REST 폴링 + WebSocket 이중 수신, 서킷 브레이커 쿨다운 설정